

Monte Carlo approximation to the distribution of MMD^2

In this workflow, we load the simulated data for each setting. Then, we compute MMD^2 for each pair of region. We then plot the histogram of MMD^2 values for each pair of regions. The pairs of regions are defined as follows: within-groups if both pairs are from Region 1 or both pairs are from Region 2, and between-groups if one pair is from Region 1 and the other pair is from Region 2.

We can change the kernel and corresponding parameters to see how the distribution of MMD^2 values changes. Go to **Define kernel** section to change the kernel and its parameters. The default kernel is inverse multiquadratic (IMQ) kernel with parameter $c = 1$.

Loading libraries

```
library(sf)
library(ggplot2)
library(spdep)
library(Matrix)
library(MASS)
library(dplyr)
library(sp)
library(gridExtra)
library(grid)
```

Load simulated data

Saved simulated data for different eta (spatial correlation), p (number of variables), n (number of spatial locations).

```

p <- 10 # 5 or 10
eta <- 0.8 # 0.3 or 0.8
eta_1 <- eta
eta_2 <- eta
n <- 2500 # 2500 or 4900
setting_n <- paste0("eta_", eta, "_p_", p, "_n_", n)
filen <- paste0("../16-MMD-properties/sim_", setting_n, ".RData")
# setting_n <- paste0("eta_", eta_1, "_eta2_", eta_2, "_p_", p, "_n_", n)
# filen <- paste0("../16-MMD-properties/", "sim_eta1_", eta_1, "_eta2_", eta_2, "_p_", p, "_n_", n, "

```

```

load(filen)

```

Define kernel

IMQ

```

imq_kernel <- function(c = 1) {
  function(D) {
    1 / (1 + (D^2 / c^2))
  }
}
k_c = 1
kernel_fn <- imq_kernel(c = k_c)
plot_title <- paste0("IMQ Kernel (c =", k_c, ")", setting_n)

filename <- paste0("mmd_imq_c_", k_c, "_", setting_n, ".png")

```

Gaussian kernel

```

# rbf_kernel <- function(sigma = 1) {
#   function(D) {
#     exp(-(D^2) / (2 * sigma^2))
#   }
# }
#
# k_c <- 2
# kernel_fn <- rbf_kernel(sigma = k_c)
#

```

```
# plot_title <- paste0("Gaussian kernel (sigma =", k_c, ")", setting_n)
#
# filename <- paste0("mmd_Gaussian_sigma_", k_c, "_", setting_n, ".png")
```

Laplacian kernel

```
# laplacian_kernel <- function(sigma = 1) {
#   function(D) {
#     exp(-D / sigma)
#   }
# }
# k_c <- 2
# kernel_fn <- laplacian_kernel(sigma = k_c)
#
# plot_title <- paste("Laplacian Kernel (sigma =", k_c, ")", setting_n)
# filename <- paste0("mmd_laplacian_sigma_", k_c, "_", setting_n, ".png")
```

Compute MMD² for all pairs of regions and plot

region 1: circle1, quad, ring

region 2: circle2, tri, circle_s

```
p <- dim(results_list[[1]])[2] - 2
```

```
source("../16-MMD-properties/mmd_sq.R")
source("../16-MMD-properties/compute_mmd_sq_pair.R")
source("../16-MMD-properties/plot_mmd_histogram.R")
```

```
region1_list <- c("circle1", "quad", "ring")
region2_list <- c("circle2", "tri", "circle_s")
```

```
region1_color <- "#E69F00" # orange
region2_color <- "#56B4E9" # blue
between_color <- "#009E73" # green
```

```

# Within-group pairs: all unordered 2-combinations
within_region1_pairs <- lapply(combn(region1_list, 2, simplify = FALSE), function(pair) {
  list(region1 = pair[1],
        region2 = pair[2],
        group = "within region1")
})

within_region2_pairs <- lapply(combn(region2_list, 2, simplify = FALSE), function(pair) {
  list(region1 = pair[1],
        region2 = pair[2],
        group = "within region2")
})

# Between-group pairs: all ordered combinations of region1 × region2
df_between <- expand.grid(region1 = region1_list,
                          region2 = region2_list,
                          stringsAsFactors = FALSE)

between_pairs <- lapply(seq_len(nrow(df_between)), function(i) {
  list(
    region1 = df_between$region1[i],
    region2 = df_between$region2[i],
    group = "between regions"
  )
})

# Combine into one list
all_region_pairs <- c(within_region1_pairs, within_region2_pairs, between_pairs)

```

```

plots <- list()

for (pair in all_region_pairs) {
  r1 <- pair$region1
  r2 <- pair$region2
  group_label <- pair$group

  # Compute MMD2 values
  mmd_values <- compute_mmd_sq_pair(r1, r2, results_list, p, kernel_fn)

  # Choose fill color
  fill_color <- switch(
    group_label,

```

```

    "within region1" = region1_color,
    "within region2" = region2_color,
    "between regions" = between_color
  )

  # Generate and store plot
  p_plot <- plot_mmd_histogram(mmd_values, r1, r2, fill_color, group_label)
  plots[[paste0(r1, "_vs_", r2)]] <- p_plot
}

```

Plots

I want to keep only one time x-axis and y-axis label

```

n_cols <- 3
n_rows <- ceiling(length(plots) / n_cols)

# Loop through and modify axis labels
plots_modified <- lapply(seq_along(plots), function(i) {
  p <- plots[[i]]

  row <- ceiling(i / n_cols)
  col <- (i - 1) %% n_cols + 1

  # Keep x-axis label only on bottom row
  if (row != n_rows) {
    p <- p + theme(axis.title.x = element_blank())
  }

  # Keep y-axis label only on first column
  if (col != 1) {
    p <- p + theme(axis.title.y = element_blank())
  }

  return(p)
})

```

```

# Open PNG device
png(filename, width = 12, height = 12, units = "in", res = 300)

```

```
# Draw the combined grid of plots
# do.call(grid.arrange, c(plots, ncol = 3, top = plot_title))
do.call(grid.arrange, c(plots_modified, ncol = n_cols, top = plot_title))
# Close device
dev.off()
```

pdf
2